

# Shiv Nadar University Chennai

|                            |                                                           |
|----------------------------|-----------------------------------------------------------|
| <b>Degree &amp; Branch</b> | B.Tech Artificial Intelligence & Data Science, Semester V |
| <b>Subject</b>             | CS3807 – Deep Learning Laboratory                         |
| <b>Name</b>                | Madhav.S                                                  |
| <b>Section</b>             | AIDS ‘B’                                                  |
| <b>Register Number</b>     | 24011101066                                               |

## Experiment 3

### Implementation of Convolutional Neural Networks (CNNs) for Image Classification

#### 1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

#### 2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

### 3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- $X$  : Input Image
- $K$  : Kernel (Filter)
- $Y$  : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

$N$     Input Size

$F$     Filter Size

$P$     Padding

$S$     Stride

#### 3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

### Identity Kernel

Preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

**Purpose:** preserves the original image; used for understanding the convolution operation.

### Sobel-X Kernel (Vertical Edge Detection)

Detects vertical edges by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

**Applications:** object detection, lane detection, face recognition.

### Sobel-Y Kernel (Horizontal Edge Detection)

Detects horizontal edges by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

**Applications:** text detection, boundary extraction, medical image analysis.

### Laplacian Kernel

Detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

**Applications:** edge enhancement, satellite image analysis, medical imaging.

### Sharpening Kernel

Enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

**Applications:** image enhancement, OCR, face recognition.

### Box Blur Kernel

Smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

**Applications:** noise removal, image smoothing, preprocessing.

### Gaussian Blur Kernel

Smooths the image while preserving overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

**Applications:** noise reduction, computer vision preprocessing, feature extraction.

### Emboss Kernel

Creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

**Applications:** texture analysis, artistic image effects.

### Outline Detection Kernel

Highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

**Applications:** image segmentation, boundary detection, shape extraction.

### Motion Blur Kernel

Simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

**Applications:** motion simulation, image restoration, video processing.

## Summary of Common Kernels

| Kernel        | Size         | Purpose                        | Applications        |
|---------------|--------------|--------------------------------|---------------------|
| Identity      | $3 \times 3$ | Preserve original image        | Baseline comparison |
| Sobel-X       | $3 \times 3$ | Detect vertical edges          | Object detection    |
| Sobel-Y       | $3 \times 3$ | Detect horizontal edges        | Boundary detection  |
| Laplacian     | $3 \times 3$ | Detect edges in all directions | Medical imaging     |
| Sharpen       | $3 \times 3$ | Enhance details                | Image enhancement   |
| Box Blur      | $3 \times 3$ | Smooth image                   | Noise removal       |
| Gaussian Blur | $3 \times 3$ | Reduce noise                   | Computer vision     |
| Emboss        | $3 \times 3$ | 3D effect                      | Artistic filtering  |
| Outline       | $3 \times 3$ | Boundary extraction            | Segmentation        |
| Motion Blur   | $5 \times 5$ | Simulate motion                | Video processing    |

## 4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

First position:  $1(1) + 2(0) + 4(0) + 5(1) = 6$

Second position:  $2(1) + 3(0) + 5(0) + 6(1) = 8$

Third position:  $4(1) + 5(0) + 7(0) + 8(1) = 12$

Fourth position:  $5(1) + 6(0) + 8(0) + 9(1) = 14$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

## 5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a  $2 \times 2$  max pooling filter:

$$\begin{array}{ll} \text{Window 1: } \begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8 & \text{Window 2: } \begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3 \\ \text{Window 3: } \begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6 & \text{Window 4: } \begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9 \end{array}$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

## 6. Numerical Example 3: Parameter Calculation

Suppose the input image is  $32 \times 32 \times 3$ , with a convolution layer of 16 filters, kernel size  $3 \times 3$ .

$$\text{Parameters} = (3 \times 3 \times 3 + 1) \times 16 = (27 + 1) \times 16 = 448$$

Therefore, total trainable parameters = 448.

## 7. Dataset

**Dataset:** CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size :  $32 \times 32 \times 3$

All ten classes were confirmed exactly balanced at 5,000 images each in the training set (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck), with no missing values.

## 8. Experimental Procedure

### Task 1 – Dataset Loading

CIFAR-10 was loaded via the Keras dataset API. Ten sample images were displayed, dataset dimensions were printed and verified (50,000 training / 10,000 test images,  $32 \times 32 \times 3$ ), and the class distribution was plotted.

### Task 2 – Convolution Layer, Kernel Size Comparison

A single convolution layer was applied to a sample image using three kernel sizes –  $3 \times 3$ ,  $5 \times 5$ , and  $7 \times 7$  – with ‘valid’ padding and stride 1, recording the resulting feature map size for each.

### Task 3 – Stride and Padding

The same sample image was convolved with stride 1 vs. stride 2, and with ‘same’ vs. ‘valid’ padding, computing the output dimensions for each combination directly from the formula in Section 3.

### Task 4 – Feature Map Visualization

Intermediate activations were extracted from every convolutional layer of the trained CNN for a single test image, displaying 8 feature maps per layer.

### Task 5 – Pooling Comparison

Max Pooling and Average Pooling were compared on both output size (identical for both, since pooling size/stride alone determines spatial shape) and classification accuracy (trained on identical smaller CNNs under matched conditions).

### Task 6 – CNN Construction and Training

Per the manual’s specification, a CNN with the architecture

Input  $\rightarrow$  Conv  $\rightarrow$  ReLU  $\rightarrow$  MaxPool  $\rightarrow$  Conv  $\rightarrow$  ReLU  $\rightarrow$  MaxPool  $\rightarrow$  Flatten  $\rightarrow$  Dense  $\rightarrow$  Softmax

was built and trained with the Adam optimizer, 20 epochs, batch size 32. *Note: alongside this manual-specification model, a larger, more elaborate CNN (5 convolutional layers across 3 blocks, with batch normalization, dropout, data augmentation, and a learning-rate schedule) was*



*also built and trained, and is used for the main evaluation, feature map visualization, and pooling/hyperparameter studies throughout this report. Both models' results are reported separately and labelled clearly below, since they answer different parts of the manual.*

## Task 7 – Evaluation

Both models were evaluated using accuracy, precision, recall, F1-score, a full classification report, and a confusion matrix.

## 9. Source Code

The complete implementation, exactly as run in Colab, is hosted on GitHub.

[https://github.com/smadhav2024/Deep\\_Learning\\_Lab/tree/main/Lab%203](https://github.com/smadhav2024/Deep_Learning_Lab/tree/main/Lab%203)

## 10. Results

### 10.1 Main CNN – Architecture and Parameters

Table 1: Trainable Parameters of Convolutional Layers (Main CNN)

| Layer | Output Shape | Params |
|-------|--------------|--------|
| conv1 | (32, 32, 32) | 896    |
| conv2 | (32, 32, 32) | 9,248  |
| conv3 | (16, 16, 64) | 18,496 |
| conv4 | (16, 16, 64) | 36,928 |
| conv5 | (8, 8, 128)  | 73,856 |

**Total Convolutional Layer Parameters:** 139,424    **Total Model Parameters:** 668,842

A manual check for conv1 confirms the formula from Section 6:  $(3 \times 3 \times 3 + 1) \times 32 = 896$ , matching Keras's own count exactly.

### 10.2 Main CNN – Training and Test Performance

The model was trained for 30 epochs at an initial learning rate of 0.001, batch size 64, with data augmentation and `ReduceLROnPlateau` monitoring validation loss (factor 0.5, patience 3). The learning rate was reduced three times over training – at epoch 14 ( $0.001 \rightarrow 0.0005$ ), epoch 20 ( $0.0005 \rightarrow 0.00025$ ), and epoch 25 ( $0.00025 \rightarrow 0.000125$ ).

Table 2: Main CNN – Test Set Performance

| Metric               | Value  |
|----------------------|--------|
| Test Loss            | 0.5095 |
| Accuracy             | 0.8257 |
| Precision (weighted) | 0.8341 |
| Recall (weighted)    | 0.8257 |
| F1-score (weighted)  | 0.8235 |

Table 3: Per-Class Classification Report (Main CNN)

| Class      | Precision | Recall | F1-score |
|------------|-----------|--------|----------|
| airplane   | 0.83      | 0.87   | 0.85     |
| automobile | 0.91      | 0.95   | 0.93     |
| bird       | 0.86      | 0.69   | 0.77     |
| cat        | 0.78      | 0.61   | 0.68     |
| deer       | 0.79      | 0.82   | 0.80     |
| dog        | 0.85      | 0.66   | 0.75     |
| frog       | 0.65      | 0.96   | 0.78     |
| horse      | 0.88      | 0.85   | 0.87     |
| ship       | 0.92      | 0.92   | 0.92     |
| truck      | 0.87      | 0.92   | 0.89     |

Vehicle classes (automobile, ship, truck) all cross 0.89 F1, while **cat** is the weakest class by recall (0.61) and **frog** shows the opposite pattern – very high recall (0.96) but the lowest precision of any class (0.65) – suggesting the model over-predicts frog somewhat, likely absorbing some of the missed cats and dogs. This cat/dog/frog overlap is a well-known difficulty in CIFAR-10 at this resolution.

### 10.3 Manual-Specification CNN (Task 6) – Performance

Table 4: Manual-Specification CNN – Test Set Performance

| Metric               | Value  |
|----------------------|--------|
| Test Loss            | 1.4740 |
| Accuracy             | 0.6587 |
| Precision (weighted) | 0.6701 |
| Recall (weighted)    | 0.6587 |
| F1-score (weighted)  | 0.6606 |
| Total Parameters     | 60,362 |

Table 5: Per-Class Classification Report (Manual-Specification CNN)

| Class      | Precision | Recall | F1-score |
|------------|-----------|--------|----------|
| airplane   | 0.65      | 0.77   | 0.71     |
| automobile | 0.84      | 0.73   | 0.78     |
| bird       | 0.59      | 0.48   | 0.53     |
| cat        | 0.46      | 0.51   | 0.48     |
| deer       | 0.61      | 0.60   | 0.61     |
| dog        | 0.50      | 0.62   | 0.55     |
| frog       | 0.83      | 0.64   | 0.72     |
| horse      | 0.70      | 0.72   | 0.71     |
| ship       | 0.73      | 0.82   | 0.77     |
| truck      | 0.78      | 0.70   | 0.74     |

The simpler two-conv-layer architecture lands about 17 points behind the main CNN (65.87% vs. 82.57%), which makes sense given it has no batch normalization, no dropout, and roughly a ninth of the parameters (60,362 vs. 668,842). Its training log also shows a telling pattern: training accuracy climbs to 87.5% by epoch 20, while validation accuracy peaks around epoch 4 (68%) and then slowly *drops* to 67.8% by the final epoch, even as training loss keeps falling – a textbook overfitting signature, and exactly what the regularization techniques in the main CNN (Section 10.1) were added to prevent.

## 10.4 Task 2 – Kernel Size Comparison

Table 6: Effect of Kernel Size on Output Feature Map Size ( $32 \times 32$  input, stride 1, ‘valid’ padding)

| Kernel Size  | Formula              | Output Shape   |
|--------------|----------------------|----------------|
| $3 \times 3$ | $(32 - 3 + 0)/1 + 1$ | $30 \times 30$ |
| $5 \times 5$ | $(32 - 5 + 0)/1 + 1$ | $28 \times 28$ |
| $7 \times 7$ | $(32 - 7 + 0)/1 + 1$ | $26 \times 26$ |

## 10.5 Task 3 – Stride and Padding

Table 7: Effect of Stride (kernel  $3 \times 3$ , ‘valid’ padding)

| Stride | Formula              | Output Shape   |
|--------|----------------------|----------------|
| 1      | $(32 - 3 + 0)/1 + 1$ | $30 \times 30$ |
| 2      | $(32 - 3 + 0)/2 + 1$ | $15 \times 15$ |

Table 8: Effect of Padding (kernel  $3 \times 3$ , stride 1)

| Padding | Output Shape   |
|---------|----------------|
| Same    | $32 \times 32$ |
| Valid   | $30 \times 30$ |

## 10.6 Task 5 – Pooling: Output Size and Accuracy

Both Max Pooling and Average Pooling, applied to a  $32 \times 32 \times 16$  feature map with a  $2 \times 2$  pool, produce an identical  $16 \times 16 \times 16$  output – output size depends only on pool size and stride, not on which pooling operation is used.

Table 9: Max Pooling vs. Average Pooling – Test Accuracy (small CNN, 8 epochs)

| Pooling Type    | Test Accuracy |
|-----------------|---------------|
| Max Pooling     | 0.5567        |
| Average Pooling | 0.5432        |

Max Pooling comes out about 1.4 points ahead of Average Pooling under identical short-training conditions – the output shapes are indistinguishable, but the values each window produces clearly aren't, and that difference is what shows up in accuracy.

## 11. Mandatory Plots

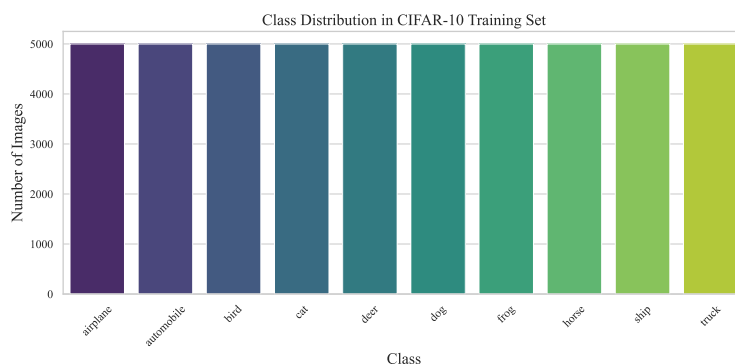


Figure 1: Class Distribution in the CIFAR-10 Training Set

**Inference:** All ten classes contain exactly 5,000 images, confirming the dataset is perfectly balanced – no resampling or class weighting was needed before training.

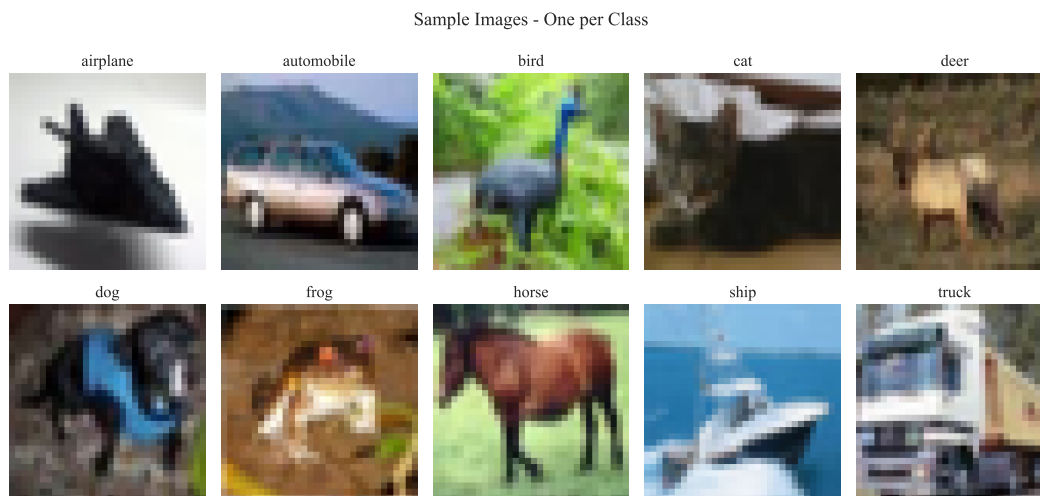


Figure 2: Sample Images – One per Class

**Inference:** Even at  $32 \times 32$  resolution, most classes are visually distinguishable, though cat, dog, and deer share enough colour and pose overlap to foreshadow the confusion seen later in both classification reports.

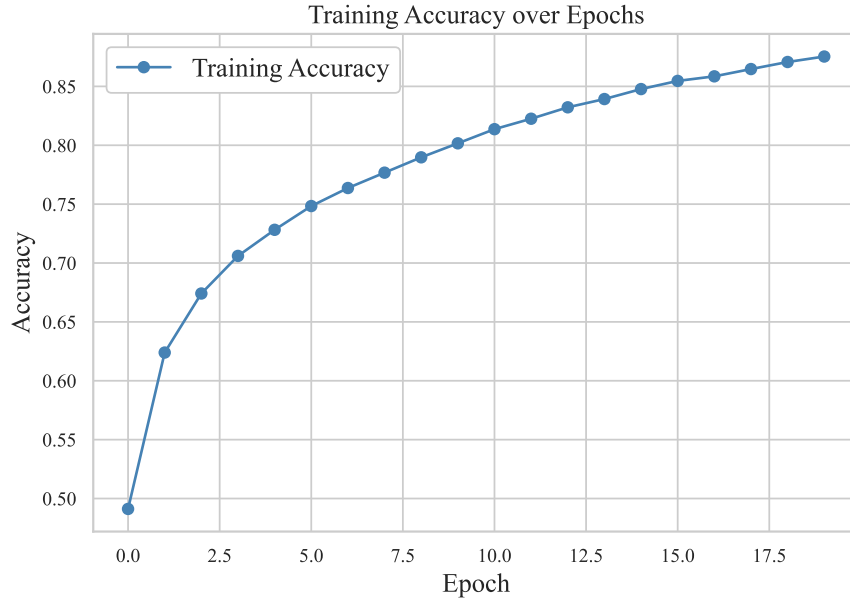


Figure 3: Training Accuracy over Epochs (Manual-Specification CNN, Task 6)

**Inference:** Training accuracy climbs steadily and almost linearly, reaching 87.5% by epoch 20 – with no dropout or batch normalization holding it back.

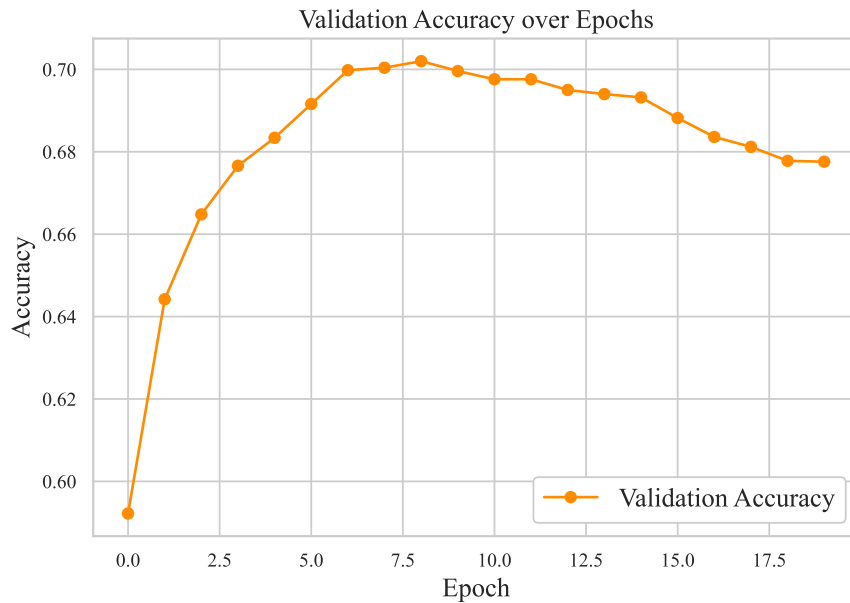


Figure 4: Validation Accuracy over Epochs (Manual-Specification CNN, Task 6)

**Inference:** Validation accuracy peaks early (around epoch 4, roughly 68%) and then gradually declines to 67.8% by epoch 20, even while training accuracy keeps rising – the gap between these two curves is overfitting, made directly visible.

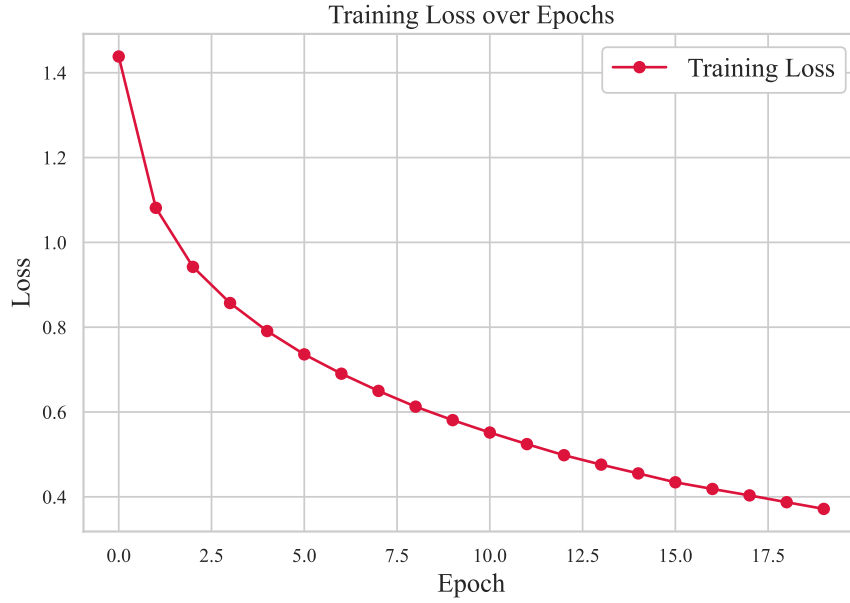


Figure 5: Training Loss over Epochs (Manual-Specification CNN, Task 6)

**Inference:** Training loss falls smoothly and continuously across all 20 epochs ( $1.44 \rightarrow 0.37$ ), with no sign of plateauing – the model keeps finding ways to fit the training data better, right up to the last epoch.

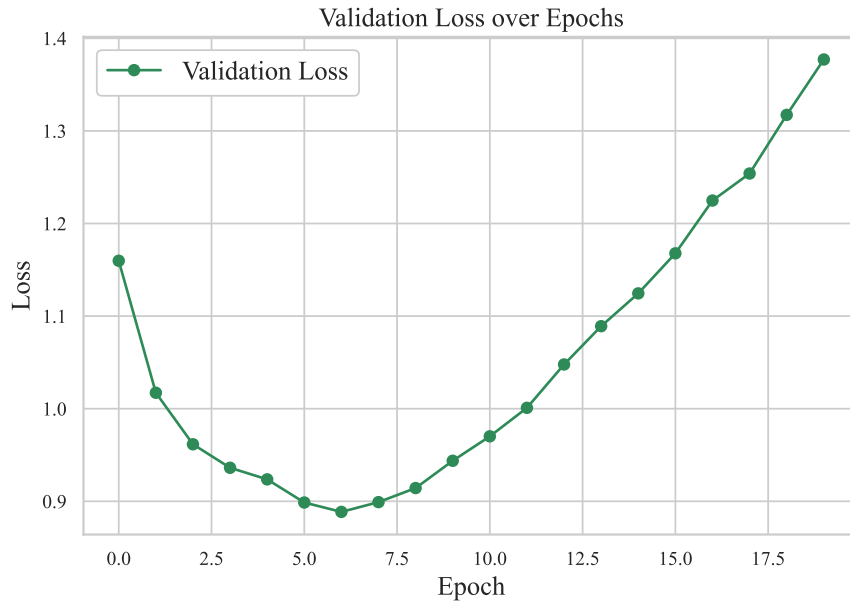


Figure 6: Validation Loss over Epochs (Manual-Specification CNN, Task 6)

**Inference:** Unlike training loss, validation loss bottoms out early (around epoch 3-4) and then climbs steadily for the rest of training, ending noticeably higher than where it started improving from – since Task 6 doesn't call for early stopping, this rise is left fully visible, and it's the clearest

single piece of evidence that this architecture overfits without regularization.

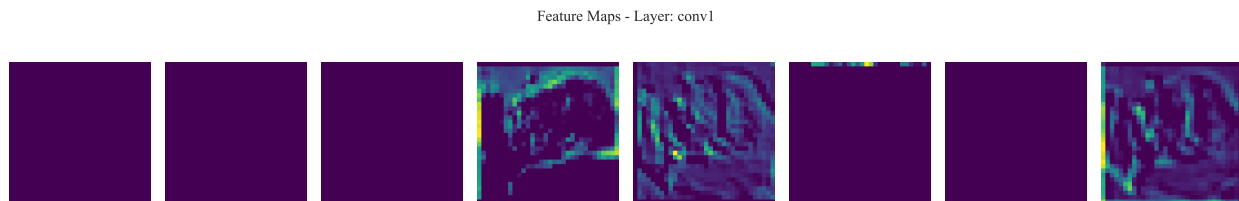


Figure 7: Feature Maps – Layer conv1 (Main CNN)

**Inference:** The earliest layer’s feature maps closely resemble the input image, picking out simple edges and coarse shapes – exactly what a first convolutional layer is expected to learn, and a direct visual match for the edge-detecting kernels discussed in Section 3.1.

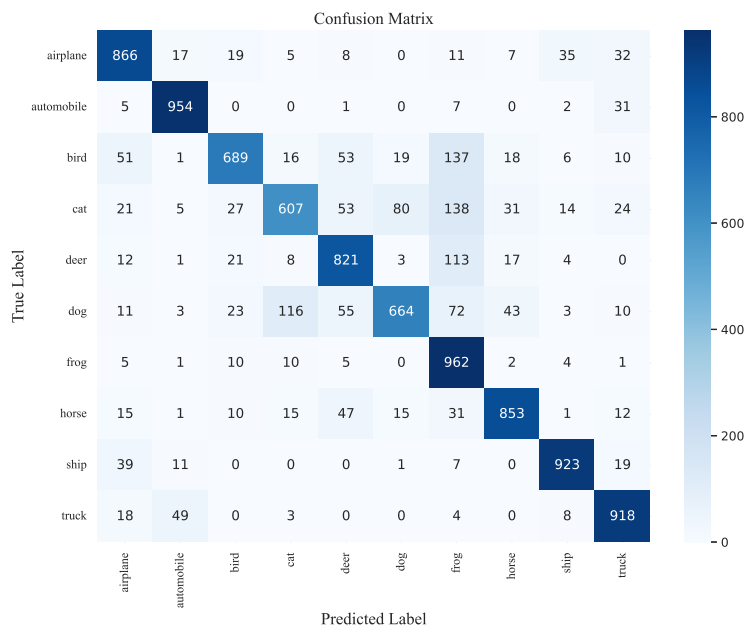


Figure 8: Confusion Matrix on the Test Set (Main CNN)

**Inference:** The diagonal clearly dominates, matching the 82.57% overall accuracy, with visible off-diagonal mass sitting specifically around the cat/dog/bird block – consistent with cat’s low recall and frog’s low precision in the classification report above.

## 12. Additional Generated Plots

Beyond the eight mandatory plots, the following were also generated during this experiment.



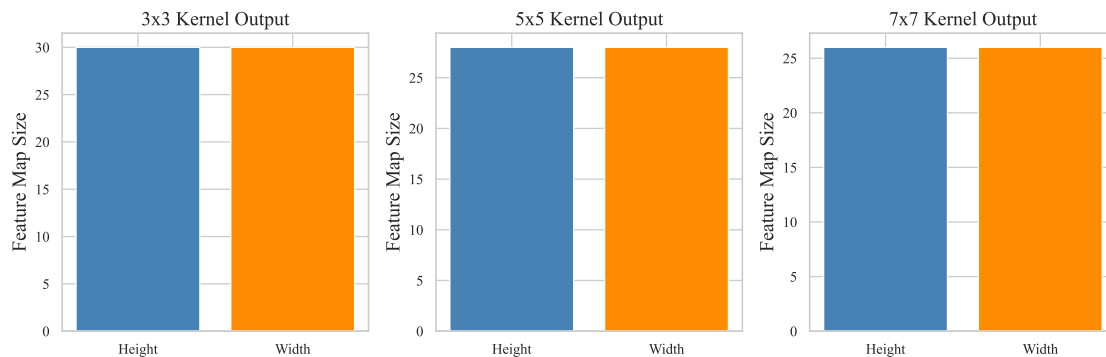


Figure 9: Output Feature Map Size for  $3 \times 3$ ,  $5 \times 5$ ,  $7 \times 7$  Kernels

**Inference:** Larger kernels shrink the output feature map faster under ‘valid’ padding – a  $7 \times 7$  kernel leaves a  $26 \times 26$  output from a  $32 \times 32$  input, while a  $3 \times 3$  kernel only shrinks it to  $30 \times 30$ . This is exactly why every convolutional layer in the main CNN uses ‘same’ padding instead – stacking several ‘valid’-padded layers would shrink the feature map away almost entirely before reaching the classifier.

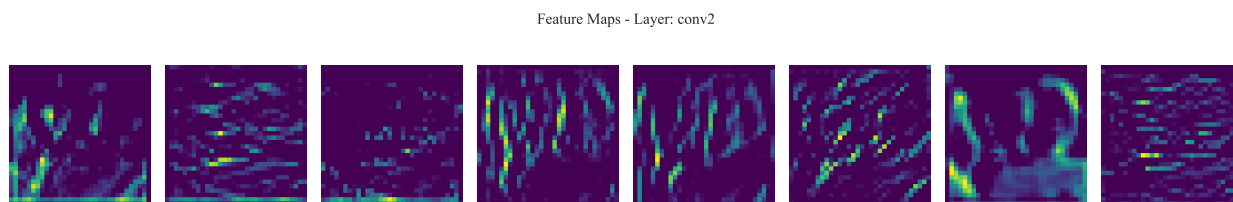


Figure 10: Feature Maps – Layer conv2 (Main CNN)

**Inference:** The second layer’s maps are already less literal than conv1’s – the object’s raw outline is fading, suggesting the network is starting to respond to specific textures and orientations rather than the whole shape.

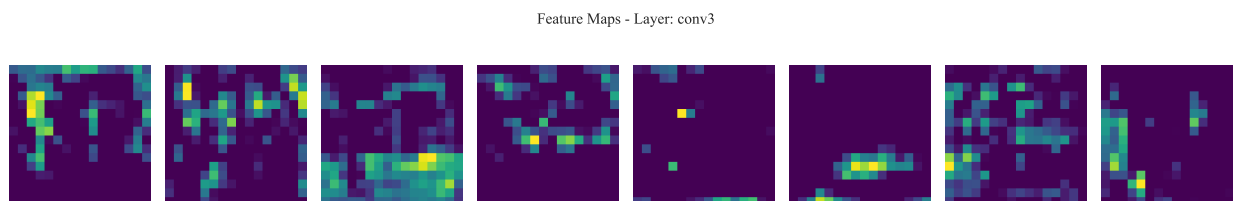


Figure 11: Feature Maps – Layer conv3 (Main CNN)

**Inference:** By the third layer, the feature maps are noticeably more abstract, and the spatial resolution has halved ( $32 \times 32 \rightarrow 16 \times 16$ ) from the first pooling step – the network is combining low-level edges into more complex, part-based patterns.

Feature Maps - Layer: conv4

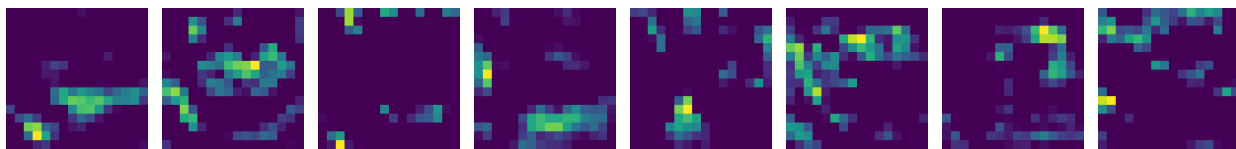


Figure 12: Feature Maps – Layer conv4 (Main CNN)

**Inference:** conv4 continues the same trend as conv3, but sparser – fewer pixels fire strongly, and the ones that do trace out specific curved or angular sub-structures rather than broad regions.

Feature Maps - Layer: conv5



Figure 13: Feature Maps – Layer conv5 (Main CNN)

**Inference:** The deepest layer's maps are sparse and highly abstract, with only a handful of filters firing strongly – the hierarchical feature learning CNNs are known for: early layers detect generic patterns, deep layers detect class-specific, high-level concepts.

Sample Test Predictions (Green = Correct, Red = Incorrect)

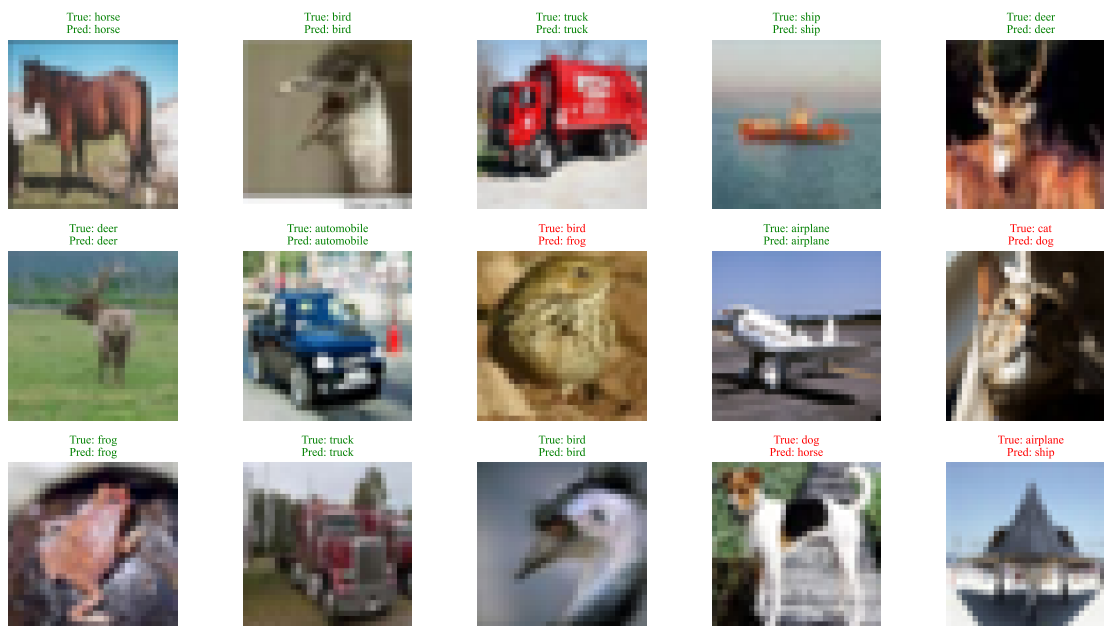


Figure 14: Sample Test Predictions (Green = Correct, Red = Incorrect, Main CNN)

**Inference:** Correct predictions dominate the sample, and the visible mistakes cluster in exactly

the classes the classification report already flagged as weaker – the qualitative examples and the quantitative metrics tell the same story.

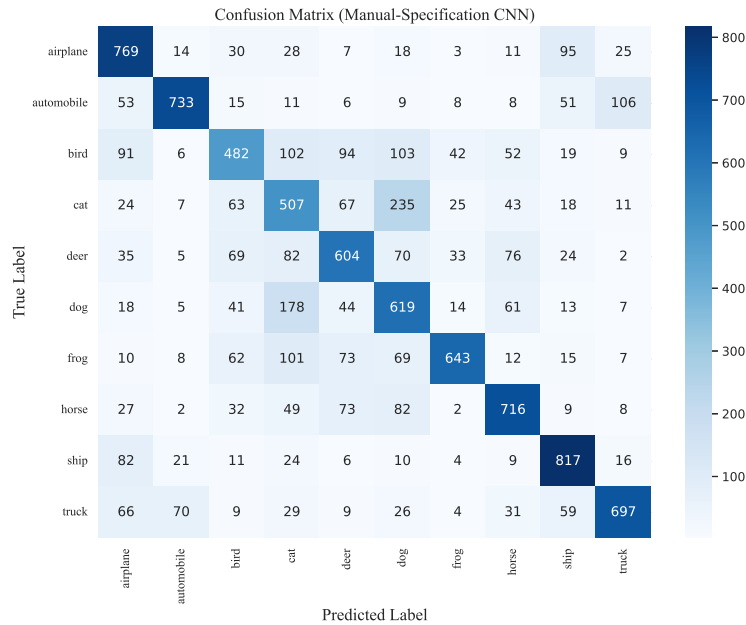


Figure 15: Confusion Matrix on the Test Set (Manual-Specification CNN, Task 6)

**Inference:** The diagonal is much weaker here than in the main CNN’s confusion matrix, matching the 17-point accuracy gap between the two models – cat and bird in particular show heavy confusion with several other classes, not just each other, reflecting this simpler model’s lower overall capacity.

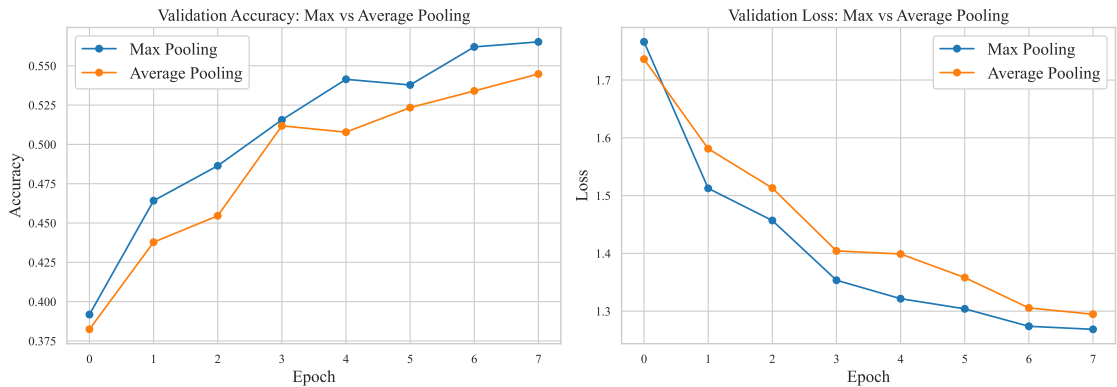


Figure 16: Validation Accuracy and Loss: Max vs. Average Pooling

**Inference:** The Max Pooling curve sits consistently at or above the Average Pooling curve across all 8 epochs, matching the final test accuracy gap (55.67% vs. 54.32%) reported in Section 10.6.

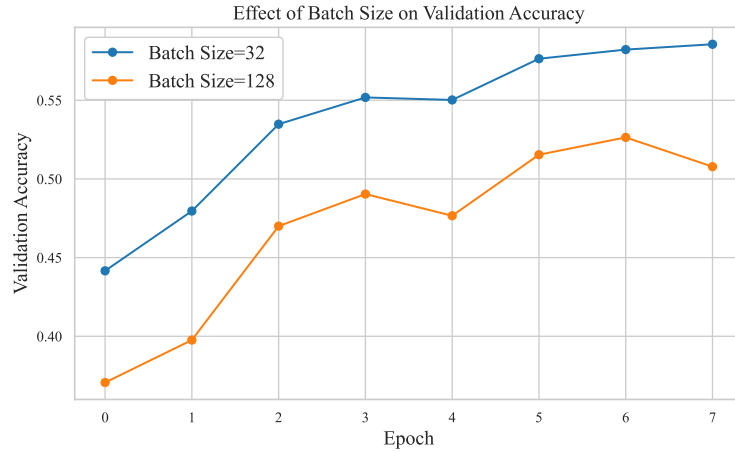


Figure 17: Effect of Batch Size on Validation Accuracy

**Inference:** Batch size 32 (58.55% test accuracy) clearly outperforms batch size 128 (51.45%) under these matched short-training conditions – more frequent, noisier weight updates appear to help this model more than the smoother, less frequent updates of the larger batch, at least within this many epochs.

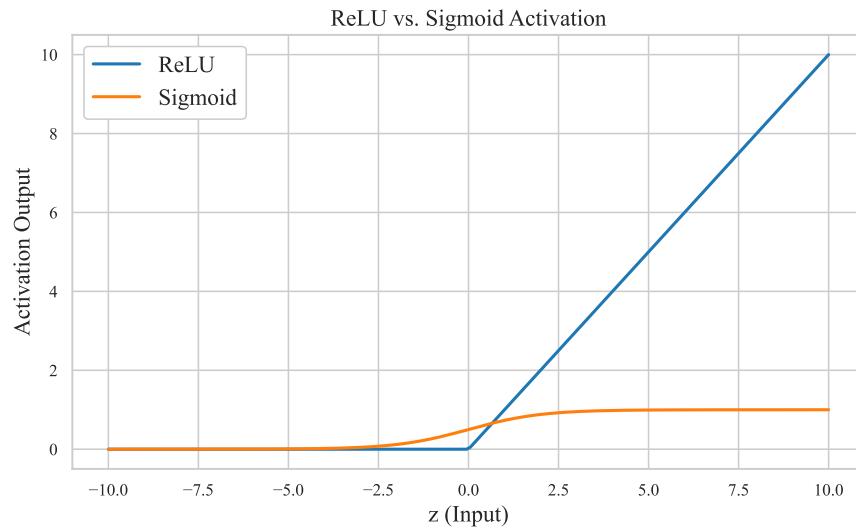


Figure 18: ReLU vs. Sigmoid Activation Functions

**Inference:** ReLU stays at zero for any negative input and passes positive values straight through, avoiding the vanishing-gradient problem that Sigmoid suffers from once its input moves far from zero – exactly why every hidden layer in both CNNs in this notebook uses ReLU rather than Sigmoid.

## 13. Additional Exercises

### 1. Output size for a $64 \times 64$ image, $5 \times 5$ kernel, stride 2, padding 2.

$$\frac{64 - 5 + 2(2)}{2} + 1 = \frac{63}{2} + 1 = 32$$

A  $64 \times 64$  input with a  $5 \times 5$  kernel, stride 2, padding 2 produces a  $32 \times 32$  output feature map.

### 2. Trainable parameters for 64 filters of size $3 \times 3$ , RGB input.

$$(3 \times 3 \times 3 + 1) \times 64 = 28 \times 64 = 1,792$$

**3. Compare ReLU and Sigmoid activation functions.** See Figure 19 (Section 12) and its inference above – ReLU is cheap to compute and avoids vanishing gradients for active neurons, making it the standard choice for hidden layers; Sigmoid squashes outputs into  $(0, 1)$  but saturates for large-magnitude inputs, which is why it’s mainly reserved for binary classification output layers rather than hidden layers in deep networks.

**4. Replace Max Pooling with Average Pooling and compare the results.** Already covered in full in Section 10.6 and Figure 18 (Section 12) – Max Pooling reached 55.67% test accuracy versus Average Pooling’s 54.32%, with identical output shapes for both.

**5. Increase filters from 16 to 64 and analyze the effect on accuracy and computation time.**

Table 10: Effect of Filter Count on Accuracy, Training Time, and Parameters

| Filters (first conv layer) | Test Accuracy | Training Time | Parameters |
|----------------------------|---------------|---------------|------------|
| 16                         | 0.5401        | 11.9s         | 25,578     |
| 64                         | 0.5929        | 15.0s         | 157,578    |

Quadrupling the filter count ( $16 \rightarrow 64$ ) improved test accuracy by about 5.3 points, but at a real cost: parameters grew more than 6-fold ( $25,578 \rightarrow 157,578$ ) and training time increased by about 26% ( $11.9\text{s} \rightarrow 15.0\text{s}$ ) for the same 8 epochs. More filters give the network a richer set of feature detectors, but that richness isn’t free – whether the accuracy gain justifies the extra time and memory depends entirely on the computational budget available.

## 14. Discussion

### 1. Why is convolution preferred over fully connected layers for images?

Convolution uses a small set of shared weights (a kernel) that slides across the whole image, so it needs far fewer parameters than a fully connected layer would to look at every pixel independently,

and it naturally captures local spatial patterns like edges and textures regardless of where they appear in the image.

## **2. How does stride affect the feature map size?**

A larger stride moves the kernel by more pixels at each step, directly shrinking the output feature map – as shown numerically in Section 10.5, stride 2 roughly halves the output size compared to stride 1 ( $30 \times 30 \rightarrow 15 \times 15$ ).

## **3. What is the role of padding?**

Padding adds extra pixels (usually zeros) around the input’s border before convolution. ‘Same’ padding keeps the output the same spatial size as the input, essential for building deep networks without the feature map shrinking to nothing after a few layers; ‘valid’ padding uses no extra pixels and lets the output shrink naturally, as shown in Section 10.5.

## **4. Why is pooling used?**

Pooling reduces the spatial size of feature maps, cutting computation and parameter count in later layers, while making the network somewhat robust to small shifts or distortions in where a feature appears.

## **5. How do feature maps represent image characteristics?**

Each feature map is the response of one learned filter across the whole image – high activation values mark where that pattern was detected. Early-layer feature maps (Section 11, conv1) look close to the original image since they detect simple edges; deeper-layer feature maps (Section 12, conv2-conv5) become progressively more abstract, as directly observed across this experiment.

## **6. Why do CNNs require fewer parameters than MLPs?**

A fully connected layer needs a separate weight for every input-output pixel pair, which explodes for anything but tiny images. A convolutional layer instead uses one small kernel shared across every spatial position, so its parameter count depends only on kernel size and filter count – not on the image’s height and width at all, as demonstrated by the 60,362-parameter Task 6 model handling the same  $32 \times 32 \times 3$  images as the 668,842-parameter main CNN.

# **15. Conclusion**

- Built and trained two CNNs on CIFAR-10: a manual-specification model (Task 6, 60,362 parameters, 65.87% test accuracy) and a larger, regularized CNN (668,842 parameters, 82.57% test accuracy) – directly demonstrating the accuracy cost of skipping batch normalization, dropout, and a deeper architecture.
- Verified the convolution output-size formula numerically across kernel size, stride, and padding (Sections 10.4-10.5), matching hand-calculated values exactly.
- Visualized feature maps across all five convolutional layers, observing the shift from edge-like, image-resembling activations in conv1 to sparse, abstract activations in conv5.

- Compared Max Pooling and Average Pooling on both output size (identical) and accuracy (Max Pooling ahead by 1.4 points).
- Directly observed the manual-specification model overfitting – validation loss rising while training loss kept falling – illustrating why the main CNN’s regularization techniques exist.
- Completed all five additional exercises, including a real accuracy/time/parameter trade-off measurement for increasing filter count from 16 to 64.

## 16. References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
4. TensorFlow Documentation: <https://www.tensorflow.org>
5. CIFAR-10 Dataset Documentation – University of Toronto.